

89. Les trois façons de faire la même chose

À la suite, par une fonction, par une méthode — et ce que chacune coûte.

1. Ce qu'il faut comprendre

À la suite, tout est dans `main` : `if (ennemiVie > 0) ennemiX++;`, avec deux variables globales. C'est le plus court, et pour **un** ennemi c'est le bon choix. Le jour où il en faut trois, il faut six variables et trois fois les mêmes lignes.

Par des fonctions : les données sont rangées dans un tableau de `struct`, le geste est nommé, et l'objet se désigne par son index — `avancer(i)`, avec `void avancer(uint8_t i) { if (troupe[i].vie > 0) troupe[i].x++; }`. Coût : un octet par argument.

Par des méthodes, ci-dessous : rigoureusement le même travail, mais le geste est écrit **là où vivent les données**, et l'objet ne se passe plus. Coût : deux octets par méthode.

Mesuré, sur ce programme : **1344 octets** à la suite, **1590** par fonctions, **1610** par méthodes. Aucune n'est meilleure dans l'absolu — la troisième rend en lisibilité ce qu'elle prend en octets, et c'est bien pour cela que le compilateur les compte et les annonce.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **673 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
démarrage          49 o ← la console, à l'allumage - la mise en route de la console
Ennemi::avancer()   32 o ← main() - ta fonction
Ennemi::blesser()   36 o ← main() - ta fonction
main()              207 o ← démarrage - ta fonction
VBlank              8 o ← la console, à chaque image - compte les images, 60 fois par seconde
AttendreImage       28 o ← image() - attend l'image suivante
AttendreVBlank      16 o ← démarrage - attend que l'écran soit libre
AttendreAcces       12 o ← EcrireNombre - attend de pouvoir écrire une case
AdresseCase         10 o ← nombre() - trouve l'adresse d'une case de l'écran
EcrireNombre        33 o ← nombre() - écrit un nombre en chiffres
CopierTuiles        31 o ← démarrage - copie les lettres et les dessins en mémoire vidéo
EffacerCarte        14 o ← démarrage - vide les deux cartes de l'écran
LireManette         37 o ← bouton() - lit les huit boutons
Tuiles              160 o ← CopierTuiles - 10 lettres sur 43
Lettres gravées : 0123456789
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EffacerCases, EcrireTexte, Diviser, Reste, Multiplier,
DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
DonneesTransfert, Notes
Fonctions du langage appelées :
```

```

nombre() × 1    36 o sur place, dans main() + AdresseCase (10 o), EcrireNombre (33 o)
bouton() × 1    12 o sur place, dans main() + LireManette (37 o)
image() × 1     3 o sur place, dans main() + AttendreImage (28 o)

```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```

// CE PROGRAMME : trois ennemis avancent ; chaque appui sur A les blesse tous.
// Au bout de 3 appuis, ils n'ont plus de vie et s'arrêtent.
//
// Chaque ennemi sait lui-même avancer et être blessé (deux méthodes).
// Les règles (« on n'avance que vivant », « la vie ne passe pas sous 0 »)
// sont écrites une seule fois, dans la struct.

#include <bouton> // lit un bouton de la manette
#include <nombre> // écrit un nombre en chiffres

```

2. UN ENREGISTREMENT — ENNEMI

```

struct Ennemi {
    uint8_t x, vie; // sa place et sa vie

    void avancer() { if (vie > 0) x++; } // avance d'un pas, seulement s'il est vivant
    void blesser() { if (vie > 0) vie--; } // perd une vie, sans passer sous 0
};

```

3. UN TABLEAU — TROUPE

```

Ennemi troupe[3]; // 3 ennemis.

```

4. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```

int main() { // Le jeu commence ici.
    for (uint8_t i = 0; i < 3; i++) { troupe[i].x = 20 + i * 40; troupe[i].vie = 3; }
    // x = 20, 60, 100 ; 3 vies chacun.
    uint8_t avant = 0; // A au tour d'avant.

    while (true) { // Répète pour toujours :
        uint8_t appui = bouton(A); // A maintenant ?
        for (uint8_t i = 0; i < 3; i++) { // Pour chaque ennemi :
            troupe[i].avancer(); // il avance (s'il vit),
            if (appui && !avant) troupe[i].blesser(); // au début d'un appui : il est blessé.
        }
        avant = appui; // On retient A pour le tour suivant.

        for (uint8_t i = 0; i < 3; i++) nombre(2 + i * 6, 8, troupe[i].x); // Affiche les 3 x.
        image(); // Attend l'image suivante.
    }
}

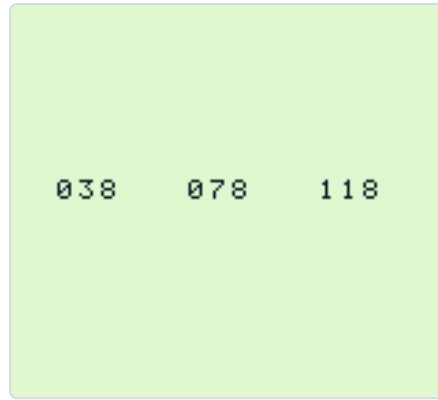
```

3. Ce qui se passe, étape par étape

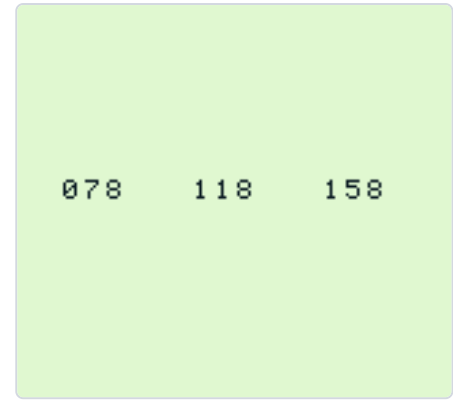
La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



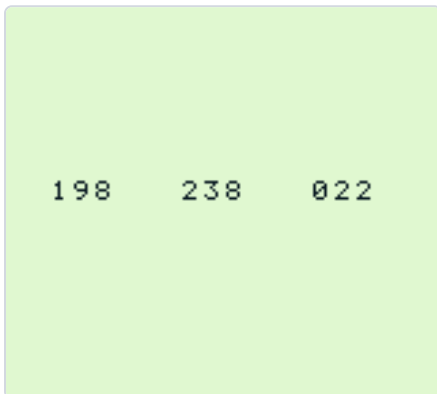
la toute première image — la console vient de démarrer — image 1



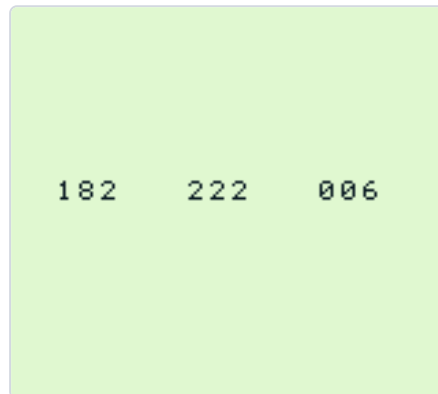
un tiers de seconde plus tard : le premier dessin est posé — image 20



au bout d'une seconde — image 60



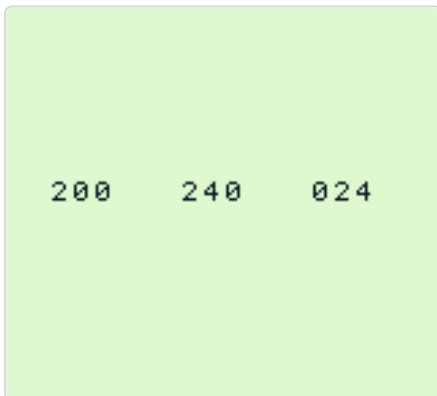
au bout de trois secondes — image 180



au bout de sept secondes — image 420

4. Ce que les touches changent

Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



A — l'écran a changé

5. Ce qu'on doit voir

Trois nombres qui montent ; A les blesse tous les trois.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ les trois avancent (038 → 068)

✓ trois appuis les tuent

✓ et ils s'arrêtent alors

✓ les trois se comportent pareil

Ensuite : **90. Une méthode qui en appelle une autre** — chapitre 13 : Ce qui va ensemble. · gameboy3 — écrire une cartouche

Game Boy en C++